

Lab #2

Fernanda Lugo

Kristine Nguyen

Xuan Phat Tran

1. Write a program in any language that converts a floating-point number to its simplified floating-point representation with excess-15 as discussed in the slides.

- The simplified model uses 1 bit for the sign, 5 bits for the exponent, and 8 bits for the significand.
- Include a header and add comments throughout your code.
- Compile and run your program, demonstrating its output with at least two examples (one positive and one negative number).
- Try these inputs: 55, 78.5, 0.01234, -234.56, and -0.4789. Also try an input with illegal characters such as A or %.

```

# -----
# Name: Fernanda Lugo
# Lab: Simplified Floating-Point Representation
# Question 1
#
# Description:
# This program converts a decimal floating-point number into
# a simplified floating-point representation using:
# 1 sign bit, 5 exponent bits with Excess-15,
# and 8 significand bits.
# -----

def convert_float(number):

    # Determine sign bit
    if number < 0:
        sign_bit = "1"
        number = abs(number)
    else:
        sign_bit = "0"

    # Handle zero
    if number == 0:
        return "0 00000 00000000"

    # Normalize the number into the form 1.xxxxx x 2^exponent
    exponent = 0

    while number >= 2:
        number = number / 2
        exponent += 1

    while number < 1:
        number = number * 2
        exponent -= 1

    # Add Excess-15 bias to exponent
    biased_exponent = exponent + 15

    # Make sure exponent fits in 5 bits
    if biased_exponent < 0 or biased_exponent > 31:
        return "ERROR: Number is out of range."

    # Convert exponent to 5-bit binary
    exponent_bits = format(biased_exponent, "05b")

    # Remove the leading 1 from normalized number
    fraction = number - 1

    # Create 8 significand bits
    significand_bits = ""

```

```

        for i in range(8):
            fraction = fraction * 2

            if fraction >= 1:
                significand_bits += "1"
                fraction -= 1
            else:
                significand_bits += "0"

    # Combine sign, exponent, and significand
    return sign_bit + " " + exponent_bits + " " + significand_bits

# Ask the user for input
user_input = input("Enter a floating-point number: ")

# Check for illegal input
try:
    number = float(user_input)

    result = convert_float(number)

    print("Simplified floating-point representation:")
    print(result)

except ValueError:
    print("ERROR: Illegal input. Please enter a valid number.")

```

Legal Input

```
PS C:\Users\lenovo\Desktop\nextgen\Lab-1-Computer-System> & C:\Users\lenovo\AppData\Local\Programs\Python\Python38-64\python.exe c:/Users/lenovo/Desktop/nextgen/Lab-1-Computer-System/Lab#2_Group_Assignment.py
Enter a floating-point number: 55
Simplified floating-point representation:
0 10100 10111000
PS C:\Users\lenovo\Desktop\nextgen\Lab-1-Computer-System>
```

Illegal Input

```
python.exe c:/Users/lenovo/Desktop/nextgen/Lab-1-Computer-System/Lab#2_Group_Assignment.py
Enter a floating-point number: 78.5
Simplified floating-point representation:
0 10101 00111010
```

```
Enter a floating-point number: A
ERROR: Illegal input. Please enter a valid number.
```

```
Enter a floating-point number: 0.01234
Simplified floating-point representation:
0 01000 10010100
```

```
Enter a floating-point number: %
ERROR: Illegal input. Please enter a valid number.
```

```
Enter a floating-point number: -234.56
Simplified floating-point representation:
1 10110 11010101
```

```
Enter a floating-point number: -0.4789
Simplified floating-point representation:
1 01101 11101010
```

2. Write a short program in any language that converts a signed integer between signed magnitude and 2's complement. The input could be signed magnitude or 2's complement.

- Ensure your code performs basic error checking and handles both positive and negative numbers.
- Compile and run your program, and show its output with at least two examples (one positive and one negative number). The inputs have to be at least 8 bits long.

```
1  # This program aims to convert a signed integer between signed magnitude and 2's complement. The input could be signed magnitude or 2's complement
2  #Kristine Nguyen
3  #Question 2
4
5  #ask user for their input and if it returns false
6  def checkingInput(answer, answerType):
7      if ((answerType != "2C") and (answerType != "SM")):
8          print("Please input either 2C or SM for the type!")
9          return False
10     if (len(answer) != 8): # making sure its 8 characters
11         print("Your answer needs 8 characters!")
12         return False
13     if not all(bit in "01" for bit in answer): # making sure it's binary only
14         print("Your answer must contain only 0s and 1s!")
15         return False
16     return True
17
18
19 #the first bit stays the same while the rest inverts
20 def invert(answer):
21     return answer[0] + ''.join('1' if bit == '0' else '0' for bit in answer[1:])
22
23
24 def functionConversion(answer):
25     if (answer.startswith("0")): #the answer will be positive for both SM and 2's Complement
26         return answer
27
28     invertedanswer = invert(answer)
29
30     carryOver = 1
31     resultantString = ""
32     for i in range(len(invertedanswer) - 1, -1, -1): # will loop through every character except for the first
33         bit = int(invertedanswer[i]) + carryOver # adding one to the integer value of the bit
34         #carry over if there is any
35         if bit == 1:
36             carryOver = 0
37         elif bit == 2:
38             bit = bit % 2
39             carryOver = 1
40
41         resultantString = str(bit) + resultantString
42
43     return resultantString
44
```

```
def askInput():  
    #ask the user for their input if it returns true  
    while (True):  
        answer = input("Input an 8-bit binary string!: ")  
        answerType = input("Is this [2C] 2's complement or [SM] Signed magnitude?:").strip().upper()  
        isValid = checkingInput(answer, answerType)  
        if (isValid == True):  
            break  
        convertedValue = functionConversion(answer)  
        print(convertedValue)  
  
askInput()
```

```
Input an 8-bit binary string!: 10000101  
Is this [2C] 2's complement or [SM] Signed magnitude?:sm  
11111011  
PS C:\Users\lenovo\Desktop\nextgen\Lab-1-Computer-System>
```

```
Input an 8-bit binary string!: 00000101  
Is this [2C] 2's complement or [SM] Signed magnitude?:sm  
00000101
```

```
Input an 8-bit binary string!: 00000101  
Is this [2C] 2's complement or [SM] Signed magnitude?:2c  
00000101
```

```
Input an 8-bit binary string!: 11111011  
Is this [2C] 2's complement or [SM] Signed magnitude?:2c  
10000101
```

3. Download Logic Friday on your computer: [Logic Friday](#) 

Logic Friday works on Windows machine only. If you need access to a Windows machine, you can use the laptops in the lab.

Here is a useful web page to get some info about the tool. <https://electronicsforu.com/buyers-guides/software-buyers-guide/logic-friday-digital-logic-design> 

Create the basic truth table for these gates:

- 4 input NAND gate (4 inputs and 1 output)
- 2 input XNOR gate (2 inputs and 1 output)
- 3 input OR gate (3 inputs and 1 output)
- a multiplexer (2 inputs, 1 select and 1 output)

<https://en.wikipedia.org/wiki/Multiplexer>

Use the tool create the equation and gates for you

Xuan will response for this
part

Logic Friday

File Operation Truthtable Equation Gates View Help

Funci...	Inputs	Outputs	True	False	DC	PI	Gates
F0	2	1	2	2	0	2	2

A	B	=>	F0
1	1		1
0	0		1

Entered by truthtable:
 $F0 = A' B' + A B;$

Minimized:
 $F0 = A B + A' B';$

Logic Friday

File Operation Truthtable Equation Gates View Help

Funci...	Inputs	Outputs	True	False	DC	PI	Gates
Y	4	1	15	1	0	4	1

A	B	C	D	=>	Y
X	X	X	0		1
X	X	0	X		1
X	0	X	X		1
0	X	X	X		1

Entered by truthtable:
 $Y = A' B' C' D' + A' B' C' D + A' B' C D' + A' B' C D + A' B C' D' + A' B C D' + A' B C D + A B' C' D' + A B' C' D + A B' C D' + A B' C D + A B C' D' + A B C D' + A B C D + A B C D;$

Minimized:
 $Y = D' + C' + B' + A';$

Logic Friday

File Operation Truthtable Equation Gates View Help

Funci...	Inputs	Outputs	True	False	DC	PI	Gates
Y	3	1	7	1	0	3	1

A	B	C	=>	Y
1	X	X		1
X	1	X		1
X	X	1		1

Entered by truthtable:
 $Y = A' B' C + A' B C' + A' B C + A B' C' + A B' C + A B C' + A B C;$

Minimized:
 $Y = A + B + C;$

Logic Friday

File Operation Truthtable Equation Gates View Help

Funci...	Inputs	Outputs	True	False	DC	PI	Gates
Y	3	1	4	4	0	2	1

S	I0	I1	=>	Y
0	1	X		1
1	X	1		1

Entered by truthtable:
 $Y = S' I0 I1' + S' I0 I1 + S I0' I1 + S I0 I1;$

Minimized:
 $Y = S' I0 + S I1;$